Clarity: Personal Finance Management for Android

Module: SE3092, Platform Based Development — Assignment 01

Working product name: Clarity

Team: IT23585294 – P.S.D. Hewage

IT23619012 – Oshan Amarasinghe

IT23680470 – G.D.C. Dabarera

1. Problem definition and objectives

1.1 Problem statement

Kavindu earns from several sources but lacks a trustworthy, low-effort view of received income, spending patterns and actual progress toward his MacBook. Existing tools fragmented or misrepresented his finances, required too much maintenance and became discouraging after missed entries. The problem is financial visibility and continued use, not merely a missing expense form.

1.2 Primary user and stakeholders

Kavindu is 25, a junior software engineer in Colombo. His salary varies between LKR 120,000 and 140,000; freelance receipts are irregular; AdSense is paid in USD; crypto produces gains and losses. He has LKR 11,200 in savings and a roughly LKR 490,000 laptop target within 12 months. The persona is supplied by the brief, not interviewed by this team.

Stakeholder/actor	Interest or role
Kavindu / authenticated app user	Low-effort capture, accurate recorded totals, understandable patterns and goal progress
Student development team	Feasible implementation, maintainable code, reproducible builds and explainable decisions
Lecturer/evaluation panel	Evidence-based requirements, technical compliance and working demonstration
Firebase Authentication	External identity service
Cloud Firestore	External persistence and synchronisation service

Clients and banks provide real-world payment context but do not interact with this app. No admin or shared-household role is needed.

1.3 Product objectives

Consolidate all four income sources while distinguishing received, pending and realised trading results. Make cash, cards and secondary-account spending visible in one place without importing bank data. Show committed/discretionary spending and category amounts in plain language. Track explicitly allocated savings toward a quantified goal without inflating progress. Make recording and resuming after missed days straightforward. Preserve each user's records securely across sessions and devices with clear offline behaviour.

Success means the user can answer “What did I record as received?”, “Where did I spend?”, “What is still owed?” and “How much have I actually allocated to the goal?” It does not mean the app can certify complete finances from incomplete manual entries.

2. Analysis of the five required lenses

2.1 Income structure

Source	Scenario evidence	Required interpretation

Source	Scenario evidence	Required interpretation
Salary	Arrives around the 25th; LKR 120,000–140,000	Record actual amount and receipt date; a schedule is not proof of receipt
Freelance	LKR 20,000–90,000 per project, milestone payments, forgotten follow-ups	Link payments to a project; track agreed amount, partial receipts and outstanding balance
AdSense	USD payout; LKR amount varies with transfer conversion	Preserve original currency/amount and the actual LKR credited or confirmed historical rate
Crypto	Q1 gain, Q3 loss, unknown year-to-date net	Record signed realised results, report monthly/year-to-date net, distinguish capital transfers from earnings

A monthly estimate is not an income transaction. Received totals must use actual receipt dates, not planned dates. Historical conversions must not change with later exchange rates. Pending amounts are displayed separately. Crypto valuations and unrealised portfolio movements are outside scope.

2.2 Shared causes of failure

Previous tool	Failure	Proposed response
Google Sheet	Broken formulas, heavy entry, discouraging gaps	Derived totals, quick entry, backdating and neutral return experience
Budgeting app	USD-only, no cloud sync, one income source, rigid budget	LKR reporting, foreign-income support, cloud persistence and multiple sources; no compulsory fixed budget
Notes	Unlabelled figures became unreadable	Structured categories, dates, searchable history and useful summaries
Bank summary	Missing cash/secondary account, incorrect PickMe category	Manual capture for every payment channel and user-controlled categories

The common problem is poor fit plus effort without timely benefit. The dashboard should be useful after one entry; charts should not depend on a full month. Friendly text must not pretend that incomplete records are complete.

2.3 Financially grounded goal progress

If all LKR 11,200 is assigned to the MacBook, the remaining amount is LKR 478,800. Across 12 monthly contribution opportunities, the initial requirement is LKR 39,900 per month. Show saved, remaining, target date and required contribution visibly.

Monthly recorded surplus is a diagnostic measure, not automatic goal saving. Goal progress changes only through an opening allocation, explicit contributions and withdrawals. Spending LKR 24,000 on coffee/dining is about 4.90% of the LKR 490,000 target; this is a comparison, not a claim that all such spending could or should have been saved.

2.4 Expense capture without bank import

Dashboard → Add Expense → amount → visible category/payment defaults → Save. Amount is the only required typed field in the usual flow; category and payment channel must still have explicit visible values. Date defaults to today and can be changed. Notes are optional. No requirement to type a merchant or justify spending.

Provide Cash, Primary bank/card, Secondary bank/card and Other as initial payment-channel labels. These are labels for capture, not automatically reconciled bank accounts. Let the user correct categories and classify each record as committed or discretionary. Never classify a cash withdrawal as spending if the resulting cash purchases are also being recorded.

2.5 Minimum useful views in the first week

View	Content	Benefit

View	Content	Benefit
Dashboard	Recorded cash income, recorded expenses, cash surplus, pending income, separate crypto net and goal card	A single overview without mixing forecasts and actuals
Spending insights	Sorted category bars plus committed/discretionary amounts and percentages	Reveals coffee/dining and spending structure clearly
Income overview	Received by source, pending project balances, monthly/year-to-date crypto result	Resolves multi-source confusion and forgotten payments
Goal details	Confirmed allocation ledger, remaining target, planned contribution dates	Connects behaviour with actual progress

Use numeric labels as well as colour. Zero spending shows “No expenses recorded” instead of a meaningless chart or division by zero. Show the selected reporting period and “Based on recorded transactions”; do not infer complete coverage from days with entries.

3. Scope and priorities

Must means core for this proposed implementation or explicitly required by the brief. Should is valuable after the core works. Could is optional. These are team proposals, not a feature list supplied by the lecturer.

Core: email/password authentication, actual and pending income, basic project payment tracking, LKR-normalised foreign income, signed crypto result tracking, expense CRUD, categories and payment channels, monthly insights, one savings goal with explicit allocations, confirmed recurring expenses, secure Firestore persistence, real-time dashboard updates and resilient offline/error states.

Should: password reset, editable source/category catalogues, merchant-category memory, recurring salary prompts, overdue invoice filters and undo deletion.

Could: Google sign-in, CSV export, local reminders, multiple goals and exchange-rate suggestions.

Out of scope: automatic bank/SMS import, money movement, crypto exchange integration, live portfolio valuation, tax advice/calculation, invoice generation, payment collection, household sharing and a guaranteed safe-to-spend balance. Do not add these while core flows remain incomplete.

4. Functional requirements and acceptance criteria

ID / priority	Requirement and rationale	Acceptance criterion
FR01 Must	Register/sign in with Firebase email/password and sign out; identity is not hard-coded	A new account can sign in; wrong credentials give a readable error; sign-out removes access to prior user's screens/data
FR02 Must	Scope all private records to the authenticated owner	Unauthenticated and cross-user reads/writes are denied by rules; switching accounts shows no other user's cached records
FR03 Must	Create/edit/delete received salary, freelance and AdSense entries with source, receipt date and amount	Changing a record recomputes the correct reporting month; deletion removes its contribution once
FR04 Must	Preserve foreign original amount/currency and confirmed conversion to LKR	USD 41 at LKR 300/USD produces LKR 12,300; later rate changes leave it unchanged
FR05 Must	Store pending income separately from receipts	A pending LKR 40,000 payment never increases received totals
FR06 Must	Track project agreement, milestones and partial receipts	Agreement 80,000 and receipts 20,000 + 20,000 show received 40,000 and outstanding 40,000; repeat confirmation cannot duplicate a receipt

ID / priority	Requirement and rationale	Acceptance criterion
FR07 Must	Record signed realised crypto results with date and LKR value; show monthly and year-to-date totals	+60,000 and -25,000 produce +35,000; cash withdrawal of capital is not another profit
FR08 Must	Quick-add an expense from dashboard with amount, date, category, spending type and payment channel	A card-paid PickMe ride of 850 can be saved with optional note blank; values remain visible and correct
FR09 Must	Edit/delete/backdate transactions with appropriate validation	Moving an expense to the prior month updates both totals; zero/negative ordinary expense is rejected
FR10 Must	Provide scenario-specific categories and allow per-entry corrections	PickMe can be changed to Transport; committed/discretionary totals update immediately
FR11 Must	Browse transaction history by month and inspect underlying category entries	Selecting Coffee and dining shows only matching entries for the chosen month
FR12 Must	Display monthly cash income, expenses, cash surplus, pending income and separate crypto result	All figures follow Section 6; no-data and negative-surplus cases render correctly
FR13 Must	Show spending by category and committed/discretionary split	Categories sum to expenses; split percentages sum to 100% when spending is positive
FR14 Must	Create/edit one active goal with target, opening allocation and future deadline	User confirms initial values; scenario figures yield 2.29% saved and 478,800 remaining
FR15 Must	Maintain goal contribution and withdrawal records; derive saved total	Contribution 10,000 changes 11,200 to 21,200; a matching monthly surplus does not add another 10,000
FR16 Must	Show remaining amount, monthly requirement and reached/overdue states	Initial 12-payment plan shows 39,900; met target shows zero remaining; overdue target never divides by zero
FR17 Must	Create recurring expense templates and confirm due occurrences	Confirming rent of 34,000 creates one actual expense; ignoring a due item creates none; variable utilities can be edited before confirmation
FR18 Must	Persist in Firestore and update dashboard through real-time listeners	Confirmed remote changes appear without manual refresh; listener errors have a usable error state
FR19 Must	Support cached views and queued writes for an existing session with sync status	Three offline entries survive restart and reconnect without duplicate records; uncached data is labelled unavailable offline.
FR20 Must	Handle validation, loading, empty, authentication, network and rejected-write states	Failed synchronisation is never presented as successful; user can retry or correct input
FR21 Should	Allow source/category additions, renaming and archiving	Archive prevents future selection while preserving historical records and totals
FR22 Should	Offer password-reset email flow	User can request reset through Firebase with neutral confirmation/error feedback
FR23 Should	Remember an explicitly selected merchant category	Subsequent merchant entry suggests it; correction remains possible
FR24 Should	Highlight overdue unpaid milestones and recurring salary awaiting confirmation	Overdue status depends on due date and remaining amount; salary is not received automatically
FR25 Could	Offer user-controlled reminders/export/Google sign-in	Each is implemented and verified only after core acceptance passes

5. Non-functional requirements

These are proposed acceptance targets, not achieved results. Mandatory assignment constraints and platform compatibility are listed in Section 12.

ID	Quality requirement	Verification
NFR01	Typical expense capture is low effort	After familiarisation, at least 4 of 5 representative attempts complete within 10 seconds from dashboard, with optional note omitted; record actual results
NFR02	Useful cached feedback is prompt	On a documented test device with 1,000 records, cached dashboard loads within 2 seconds and local save feedback within 1 second in at least 9 of 10 trials
NFR03	Records survive interruption without duplicate saves	Verify offline entry, process restart, reconnect and repeated Save/Confirm; stable operation identity produces one record
NFR04	Monetary calculations are deterministic	LKR stored as integer minor units; decimal arithmetic for conversion, final rounding to two decimals; no binary floating-point money totals
NFR05	Data is isolated and passwords are not stored by the app	Security-rule tests cover user A/B, unauthenticated requests and invalid payloads; no password or service- account credentials in app data/logs
NFR06	Interface is accessible	At least 48dp interactive targets, meaningful descriptions, labelled charts, readable enlarged text and no information communicated by colour alone
NFR07	Gaps and missing records are handled honestly	Backdating is allowed; no guilt/streak-loss messages; summaries identify their dependence on recorded data
NFR08	Separation is maintainable	Review confirms no data-source calls or calculation logic in Composables; UI observes read-only state from ViewModels
NFR09	Queries are bounded and listeners managed	Month/source queries and required indexes documented; listeners cleaned up when no longer needed
NFR10	Error states are recoverable	Test empty database, denied write, expired session, network loss, invalid amount and overdue goal without crashes or blank screens

6. Business rules and calculation definitions

6.1 Reporting and currencies

Use LKR as the initial reporting currency. This is a scope decision based on the scenario, not a requirement for selectable base currencies. Store the currency code, positive decimal original amount, actual receipt/result date, LKR value, conversion basis and positive conversion rate where applicable. Crypto result direction is a separate gain/loss sign; do not represent ETH amounts as integer cents. LKR values use integer cents; original non-LKR amounts use sufficient decimal precision.

For rate-based conversion: LKR value = original amount × LKR per unit, rounded half-up to two decimals once. Where a bank-credit amount is entered, that confirmed net credit is authoritative; derive an effective rate for explanation and do not subtract fees twice. Pending conversions are estimates; receipt confirmation fixes actual values. LKR entries use rate 1.

Use transaction date in the user's reporting timezone (initially Asia/Colombo), not server upload date, to allocate a reporting month. Store audit timestamps separately. Ordinary received income/expenses must be positive and not future-dated. Planned due dates may be future dates. Crypto loss is valid; empty or zero crypto results are rejected.

6.2 Income, crypto and cash

Ordinary cash income = actual salary + received freelance payments + actual AdSense credit + any other supported ordinary cash income. Pending payments are excluded.

Crypto net = sum of signed realised trading results for the selected month/year. It excludes capital deposits, capital withdrawals and unrealised price movements. The user supplies the realised result; the app does not infer it from a sale's gross proceeds.

Combined recorded earnings result, if displayed = ordinary cash income + crypto net. Label it explicitly; it is not a bank balance or disposable cash.

Recorded cash surplus = ordinary cash income − recorded ordinary expenses. Display crypto separately so gains left in an exchange are not assumed available for daily spending. This simplified metric excludes investment-capital transfers and therefore is not a reconciliation of all cash movement. No “safe-to-spend” claim is made.

Moving existing funds between accounts, withdrawing cash or allocating existing funds to the goal is not new income or an expense. The first version does not maintain bank account balances. If transfer logging is added later, it must remain excluded from income/expense totals.

6.3 Project receipts

Project outstanding = agreed total − sum of actual project receipts. Each receipt can reference a milestone. Partial receipts remain separate actual payment records; the unpaid remainder remains pending. Planned milestone totals must not be added again on top of project outstanding. Prevent overpayment beyond the agreed total unless the agreement is explicitly revised. Receipt confirmation must be idempotent so retries do not create additional income.

6.4 Spending

Total expenses E = committed C + discretionary D. Each expense has exactly one spending type. Store the type on the record so changing a category default does not silently rewrite past analysis.

If E > 0: committed share = 100 × C/E; discretionary share = 100 × D/E. When E = 0, show no-data text. Avoid dividing by zero or presenting a percentage against negative income. Category sums must equal E.

6.5 Goal savings and monthly plan

Goal saved S = confirmed opening allocation + contributions − withdrawals. Never add monthly cash surplus automatically. Contributions represent the user's assertion that money was set aside; the app does not transfer or verify funds. Withdrawals cannot exceed recorded goal savings. Edits/deletions must leave a valid nonnegative ledger.

Remaining R = max(target T − S, 0). Progress = 100 × S/T for T > 0; cap the visual progress bar at 100% while retaining any overfunding amount.

Create a visible monthly contribution schedule when the goal is set: one due date per month starting one month after the goal start, capped to the last day when necessary, with a final opportunity on the deadline if it is not already a scheduled date. A 12-month goal has 12 opportunities. Number N is the count of future scheduled opportunities, including today. Required contribution = R/N, rounded upward to the nearest LKR cent; the last contribution can be smaller. Show the next date and N so “months remaining” is unambiguous. If N = 0 and R > 0, show overdue and request a revised deadline, not a division result.

Example: T = 490,000; opening allocation = 11,200; N = 12. S = 11,200; R = 478,800; required = 39,900; progress ≈ 2.29%.

Do not compare today's partial month to a whole-month target and label it failure. If ahead/behind status is added, compare net contributions against original planned contributions whose due dates have passed, retaining the original

baseline. Minimum core shows actual saved, remaining and revised requirement without forecasting a guaranteed completion date.

6.6 Recurrence and offline behaviour

A recurring template is an obligation, not an actual expense. Each due occurrence has a stable identity. Confirmation creates one actual payment with a user-confirmed date and amount. Cancelling a template stops future occurrences, not past payments.

Offline support assumes a previously authenticated session and locally available data. New registration/sign-in, password reset and uncached history require connectivity. Queued local records show pending status; server rejection must be visible. Define

edit conflicts and deletion reconciliation in the design phase, especially if Room is added. Clear or isolate all local data by user on account changes.

7. Conceptual information requirements

These are domain concepts, not the final Firestore collection layout.

Concept	Essential information
User preferences	Owner identity, reporting timezone, reporting currency, visible defaults
Income receipt	Stable ID, owner, source, actual date, original amount/currency, LKR amount, conversion basis, optional project/milestone, note, audit metadata
Pending income	Source, expected amount/currency, due date, project/milestone, settled amount/status; estimates distinguishable from receipts
Project/milestone	Owner, name, optional client, agreed amount, milestone amount/due date; linked actual receipts
Crypto result	Owner, gain/loss, original amount/currency, realised date, fixed LKR value, optional note
Expense	Owner, positive LKR amount, transaction date, category, stored spending type, payment channel, optional merchant/note, audit metadata
Category/source	Owner or default catalogue identity, name, default type/currency, active/archived state
Goal	Owner, name, positive target, opening allocation, start/deadline, contribution schedule
Goal allocation	Goal, owner, contribution/withdrawal, positive amount, date, note; immutable identity
Recurring template/occurrence	Owner, category, payment channel, suggested amount, cadence, due date, confirmation identity/status

8. Use cases and flows

Use case	Actors	Requirements
UC01 Authenticate	User, Firebase Authentication	FR01–02
UC02 Capture/correct an expense	User, Firestore	FR08–10, FR19–20
UC03 Record ordinary income/conversion	User, Firestore	FR03–04
UC04 Track a project and receive a partial payment	User, Firestore	FR05–06
UC05 Record/review crypto result	User, Firestore	FR07
UC06 Review monthly finances and drill down	User, Firestore	FR11–13, FR18
UC07 Set goal and record allocation	User, Firestore	FR14–16
UC08 Confirm recurring cost	User, Firestore	FR17

UC02 — Quick expense. Preconditions: authenticated session, or permitted cached session offline. User taps Add Expense, enters amount, reviews category/type/channel/date and saves. System validates, assigns a stable ID, persists or queues, displays status and updates totals. Alternative: invalid amount keeps the form with inline guidance; offline saves remain pending; a denied write

displays failure and supports correction. Postcondition: exactly one owned expense exists or a clearly identified unsynchronised record remains.

UC04 — Partial freelance receipt. User opens project, selects milestone, enters amount actually received and receipt date, confirms conversion if relevant and saves. System verifies remaining balance, records one receipt, updates pending balance and received totals. Retrying the same confirmation does not duplicate it. Unpaid balance remains visible.

UC07 — Goal allocation. User confirms goal target/deadline and opening allocation. Later they record money set aside or withdrawn. System updates the allocation ledger, saved total and required future contribution. It does not create income/expense or automatically transfer monthly surplus. Invalid withdrawal leaves the ledger unchanged.

The use case diagram is provided in Appendix A. This catalogue and the diagram describe the same actors and scope.

9. Traceability to scenario and rubric

Evidence	Derived need	Requirements/verification
S1 Section 3.2 salary variation and unknown receipts	Actual receipt records	FR03; salary calculation check
S1 Section 3.2 unpaid milestones and unmatched payments	Project linkage and pending balance	FR05–06; AT02
S1 Section 3.2 USD AdSense conversion	Fixed historical LKR credit	FR04; AT01
S1 Section 3.2 crypto gains/losses	Signed net result	FR07; AT03
S1 Section 3.3 cash and secondary-account spending	Payment-channel capture	FR08; AT05
S1 Section 3.3 no spending categories/ratio	Correctable categories and split	FR10, FR13; AT06
S1 Section 3.3 gym/subscriptions and variable utilities	Confirmed recurring items	FR17; AT09
S1 Section 3.4 490,000 goal and 11,200 savings	Explicit goal allocations and required saving	FR14–16; AT07–08
S1 Section 3.4 abandoned sheet after gap	Backdating, neutral tone	FR09; NFR07
S1 Section 3.5 rigidity, no sync, high effort	Flexible capture, offline and cloud	FR08, FR18–20; NFR01, NFR03
S1 Section 4.3 and Section 5 technical requirements	Authentication, ownership, architecture and stack	FR01-02; NFR05, NFR08; Section 12 constraints
S1 Section 6–7 and detailed rubric	Required documentation, working APK and explainable decisions	Section 13 submission components

10. Acceptance examples for later implementation

All amounts below are test fixtures, not claims of Kavindu's actual transactions. These tests are specified, not yet executed.

ID	Input/action	Expected result

ID	Input/action	Expected result
AT01	Salary 128,000; USD 41 AdSense at 300 LKR/USD	Ordinary received income 140,300; USD record retains 12,300 after rate changes
AT02	Project 80,000; pending 40,000 milestone; two receipts of 20,000	Received 40,000; outstanding 40,000; no duplicate pending amount counted as income
AT03	Crypto gain 60,000 and loss 25,000	Net +35,000; year-to-date includes both; ordinary cash income unchanged
AT04	Transfer 10,000 between own accounts or allocate 10,000 to goal	No new earnings or ordinary expense
AT05	Save 850 PickMe/card; correct category from Shopping to Transport	One expense remains; category breakdown changes; total remains 850
AT06	Committed expenses 40,000; discretionary 10,000	Total 50,000; shares 80% and 20%; zero-spend month has no percentage division
AT07	Target 490,000; opening allocation 11,200; 12 scheduled opportunities	Remaining 478,800; monthly requirement 39,900; progress about 2.29%
AT08	Same goal; cash surplus 10,000 and one explicit contribution 10,000	Saved 21,200, not 31,200; withdrawal 1,200 yields 20,000
AT09	Rent template 34,000 due; confirm once then retry	Exactly one actual expense of 34,000; unconfirmed utilities add zero actual expense
AT10	Existing session offline: create three expenses, restart, reconnect	Three records remain, synchronise once each and show accurate status
AT11	User A attempts User B's read/write; then account switch	Request denied; no prior-user private data visible
AT12	Empty account, no income, expenses above income, overdue/met goal	Useful empty states, honest negative surplus, no invalid division or crash
AT13	Return after 10-day gap; enter prior-month expense	No guilt message; record appears in chosen prior month, not upload month
AT14	Received income 1,000; recorded expenses 1,500	Cash surplus -500; display does not label it actual savings or bank balance
AT15	Original positive foreign amount with rate 0 or negative	Validation prevents save; existing valid historical records remain unchanged
AT16	Server denies an offline queued record	UI identifies unsynchronised failure and allows correction; does not claim cloud save succeeded

11. Assumptions, validation questions and risks

Question to validate	Provisional decision	Impact if changed
Is all 11,200 reserved for the laptop?	User must confirm; scenario demo may explicitly allocate it	Changes goal opening allocation and monthly requirement
Does 12 months mean from goal setup or a fixed calendar date?	From setup, with visible chosen deadline	Changes schedule and progress baseline
Are crypto gains withdrawn or left invested?	Track realised result separately from cash income	More detailed cash/investment accounting may be needed later

Question to validate	Provisional decision	Impact if changed
Can manual bank-credit values be provided for AdSense?	Accept confirmed LKR credit or manual historical rate	Could require a richer fee model
Is a payment-channel label enough?	Yes; no reconciled account balances	Account accounting would significantly broaden scope
Which categories feel committed to Kavindu?	Suggested defaults with per-entry override	Adjust labels/defaults, not historical records automatically
Are reminders wanted?	Optional and deferred	Requires permission and scheduling design if accepted
Local persistence approach	Room is strongly recommended, not mandatory. Offline persistence remains required (S1 Section 5).	Determines local persistence design effort
Presentation format	Rubric lists a deck; Section 6.4 requires a live engineering discussion. See Section 13.	Changes demo preparation deliverables
What are team size and available development devices?	Only the listed member is confirmed here. Record actual contributors and test devices; maximum three members.	Adjust ownership, schedule and validation coverage

If representative users are available, an optional interview and quick-add prototype task can validate the proposed design. Record consent, actual responses and resulting changes. Do not invent interviews or answers for the supplied composite persona.

Risk	Mitigation
Feature growth exceeds time available	Complete Must flows first; freeze optional additions before final testing
Manual entries create false confidence	Label recorded-data coverage; no guaranteed balance or affordability claims
Currency/goal errors inflate progress	Centralize rules and use AT01–08 as calculation checks
Offline retries or two caches duplicate records	Stable IDs, clear persistence ownership, reconnect tests and documented conflict policy
Team cannot explain assisted work	Review each requirement and implementation decision; practice individual explanations and disclose assistance
Missing submission element loses criterion marks	Track repository access, rules, live Firebase APK, PDF and demo attendance explicitly

12. Assignment constraints and platform compatibility

S1 means the official SE3092 Assignment 01 specification and detailed marking rubric, issued 23 September 2026. References such as “S1 Section 3.2” identify sections of that assignment, not this document.

The constraints below come from S1. The feature priorities, numerical acceptance targets and calculation rules in Sections 3–6 are proposed design decisions derived from the scenario; they are not a lecturer-supplied feature checklist.

Constraint	Required boundary and source
Language and build	100% Kotlin; no Java files or Java interop in the application layer. Use Kotlin DSL for Gradle. Source: Sections 4.3 and 5.
UI and navigation	Jetpack Compose with Material Design 3; no XML layouts. Single activity with Navigation Compose. Typed arguments are preferred. Source: Section 5.
Architecture	Strict MVVM: UI → ViewModel → Repository → Model. No business/ViewModel logic in Composables and no data-source calls from the UI. Source: Sections 4.3, 5 and rubric.

Constraint	Required boundary and source
State and concurrency	StateFlow or LiveData; no raw mutable state exposed by ViewModels. Kotlin Coroutines with viewModelScope for async operations. Source: Section 5.
Dependency injection	Hilt (preferred) or Koin. No manual ViewModel instantiation inside Composables. Source: Section 5.
Authentication	Firebase Authentication; email/password mandatory. Google Sign-In is bonus credit. No hard-coded user identity. Source: Sections 4.3 and 5.
Persistence and security	Firestore is the primary store; dashboard real-time listeners and submitted Firestore Security Rules are required. Offline persistence is assessed. Source: Sections 4.3, 5 and rubric.
Local caching	Room is strongly recommended, not stated as mandatory. The chosen approach must support offline persistence without blocking the UI when the network is unavailable. Source: Section 5.
Platform compatibility	Minimum SDK 26 (Android 8.0); target SDK 34 or higher. APK must install/run on Android 8.0+. Record the tested device/emulator and API level. Source: Sections 4.3, 5 and 6.3.
Error handling and input	Handle basic errors, network loss and empty data without crashes or blank screens. Automatic bank-data import is unavailable in the scenario. Source: Sections 4.1 and 4.3.
Team and deadline	Group assignment with a maximum of three members. Submission deadline: 30 October 2026. The listed team member is retained; additional identities have not been assumed. Source: assignment cover.

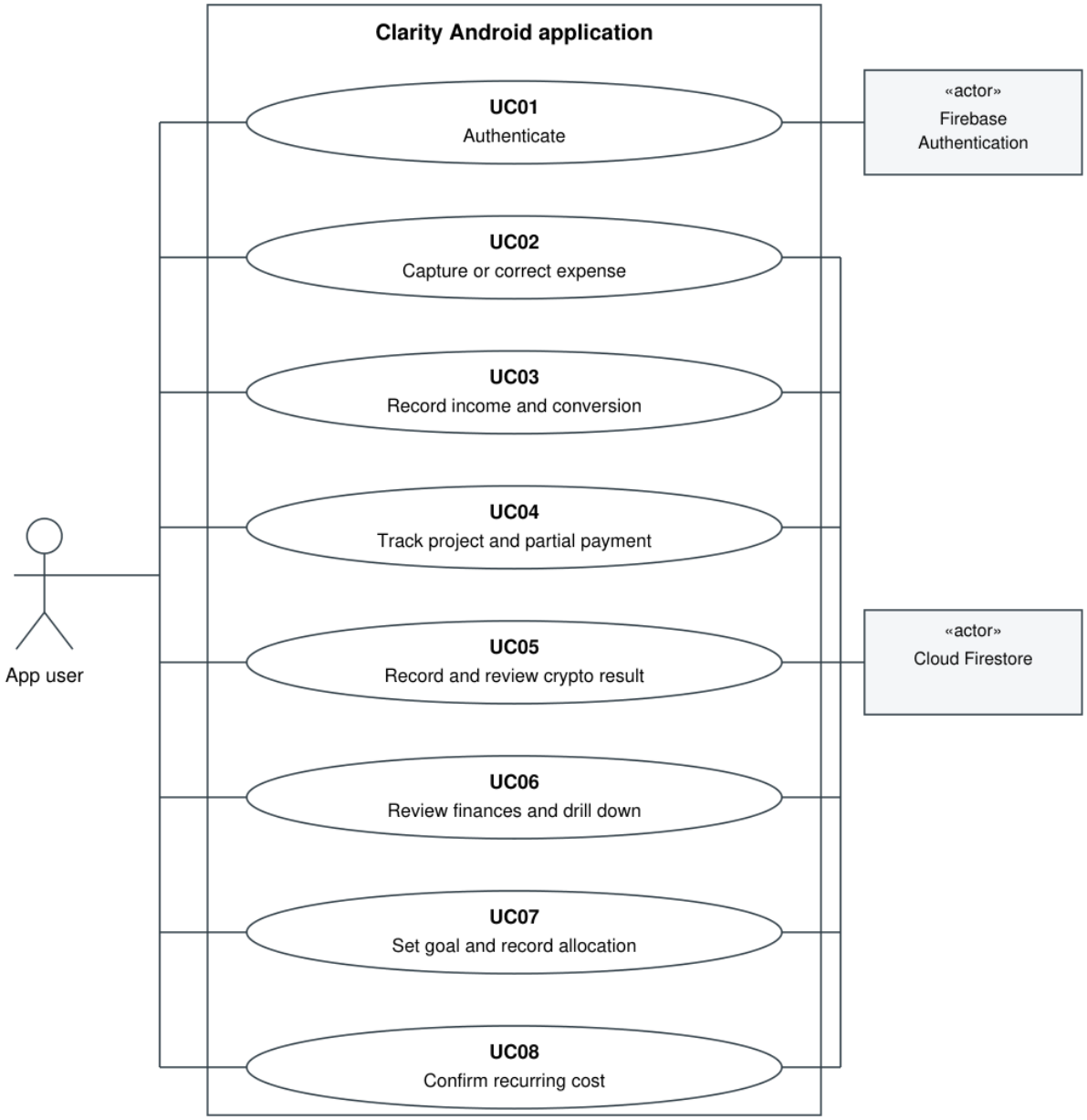
13. Required final submission components

This document covers problem analysis and requirements. It is one part of the final technical design document. Page count alone does not establish a complete submission; the design artefacts below must describe the actual implementation.

Component	Submission requirement	Source
Requirements and use cases	Requirements analysis/SRS with functional requirements, non-functional requirements and constraints. A use case diagram is an expected Criterion 1 deliverable; see Appendix A.	S1 Section 4.2; rubric p. 9
Technical design PDF	Minimum 15 pages, no maximum. Include requirements analysis, Firestore schema, MVVM diagram, at least three annotated wireframes/mockups, and design-decision justifications specific to Kavindu and the implementation.	S1 Sections 4.2 and 6.2
Firestore design	Document collection structure, indexing strategy and multi-currency handling. Provide a Firestore schema diagram. Section 7 is conceptual information, not the completed database design.	S1 Section 4.2; rubric pp. 9–10
Architecture and UI design	Map screens, ViewModels, Repositories and data sources in an MVVM diagram. Include the navigation graph and annotated UI wireframes. These artefacts are not supplied by this requirements document.	S1 Section 4.2; rubric pp. 9–10
Source repository and README	Private GitHub/GitLab access link submitted through the LMS before the deadline. Genuine iterative commit history. README: overview, Firebase configuration, build instructions and architectural summary.	S1 Section 6.1
Security rules	Submit the Firestore Security Rules file. Requirements and rule-test plans are not substitutes for the implemented rules.	S1 Sections 4.3, 5 and rubric
APK and validation	Release-signed or debug APK connected to a live Firebase project. Firebase-emulator-only configuration is not accepted. Validate on a physical device or correctly configured Android 8.0+ emulator.	S1 Section 6.3
Live demo and supporting slides	Attend the scheduled 15-minute session, run core flows live, and explain architecture, schema, queries and requirement derivation. Section 6.4 describes an engineering discussion; the detailed rubric also lists a slide deck. Prepare that supporting artefact without replacing the live demonstration.	S1 Section 6.4; rubric p. 10
Authorship and understanding	Submitted code, design decisions and documentation must represent the team's own intellectual work. AI is allowed as a learning aid; disclose AI-generated code rather than submitting it as original work.	S1 academic integrity note, p. 7

Appendix A Use case diagram

Scope: Clarity Android application. UC identifiers match Section 8. The app user is the primary actor; Firebase Authentication and Cloud Firestore are external supporting services. Associations show participation, not execution order.



Authenticated access and ownership checks apply to UC02–UC08. Firestore persistence and synchronisation support those use cases. No administrator, bank-import actor or payment-transfer service has been added.